CONCEPT

What is a Window Function?

A **window function** performs a calculation across a set of rows that are related to the current row — its "window." Unlike `GROUP BY`, which collapses many rows into one, a window function *keeps every row* and adds the computed value as a new column.

> **The core insight:** GROUP BY asks "give me one row per group." Window functions ask "give me every row, but also tell me something about the group each row belongs to."

Compare these two approaches to getting department salary averages:

GROUP BY — ROWS COLLAPSE

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department; -- Result: one row per department -- Engineering | 95000 -- Marketing | 72000
```

WINDOW FUNCTION — ROWS PRESERVED

```
SELECT name, department, salary, AVG(salary) OVER (PARTITION BY department)
AS dept_avg FROM employees; -- Result: every employee row, with their dept
average added -- Alice | Engineering | 110000 | 95000 -- Bob | Engineering |
80000 | 95000 -- Carol | Marketing | 72000 | 72000
```

The magic keyword is `OVER()`. It turns any aggregate — or a specialized function like `ROW_NUMBER()` — into a window function.

STRUCTURE

Anatomy of a Window Function

```
function_name(args) OVER (  ← the function + OVER keyword PARTITION BY col  ←
optional: divides rows into groups (like GROUP BY) ORDER BY col [ASC|DESC]  ←
optional: sets the row order within each partition ROWS BETWEEN ... AND ...  ←
optional: defines the "frame" of rows to consider )
```

The three clauses inside `OVER()` are all optional:

- **PARTITION BY** — Divides the result set into groups. The function resets for each group. Omit it to treat the entire table as one partition.
- **ORDER BY** — Determines row order within each partition. Required for ranking functions and running calculations.
- **Frame clause** (ROWS/RANGE BETWEEN) — Fine-tunes exactly which rows relative to the current row are included. Defaults vary by function.

> **Execution order matters:** Window functions run *after* WHERE, GROUP BY, and HAVING, but *before* ORDER BY and LIMIT. You cannot use a window function inside a WHERE clause — wrap it in a CTE or subquery if you need to filter on a window result.

REFERENCE

Common Window Functions

Ranking Functions

`ROW_NUMBER()`

Assigns 1, 2, 3… to each row. No ties — always unique within the partition.

`RANK()`

Like ROW_NUMBER but ties share a rank, leaving gaps (1, 2, 2, 4).

`DENSE_RANK()`

Like RANK but no gaps after ties (1, 2, 2, 3).

`NTILE(n)`

Splits rows into *n* roughly equal buckets, labeled 1 through n.

Value Functions

`LAG(col, n)`

Returns the value from *n* rows before the current row (default n=1).

`LEAD(col, n)`

Returns the value from *n* rows after the current row.

`FIRST_VALUE(col)`

Returns the first value in the window frame.

`LAST_VALUE(col)`

Returns the last value in the window frame (beware default frame!).

Aggregate Functions as Windows

`SUM() OVER()`

Running or partitioned sum without collapsing rows.

`AVG() OVER()`

Running or partitioned average alongside each row.

`COUNT() OVER()`

Running or partitioned count per row.

`MAX() / MIN() OVER()`

Partition-level max or min visible on every row.

WORKED EXAMPLE

Your First Window Function

Scenario: You have a table `sales` with columns `id, rep_name, region, amount, sale_date`. You want every sale row, but with each rep's rank within their region based on amount.

```
SELECT rep_name, region, amount, RANK() OVER ( PARTITION BY region -- restart ranking per region ORDER BY amount DESC -- highest amount = rank 1 ) AS region_rank FROM sales; -- Result: every row, with a rank added -- Alice | West | 50000 | 1 -- Bob | West | 42000 | 2 -- Carol | East | 61000 | 1 -- Dave | East | 61000 | 1 ← tie: both rank 1 -- Eve | East | 35000 | 3 ← rank 3, not 2 (gap after tie)
```

What happened:

- `RANK()` is the window function — it computes a ranking number.
- `OVER()` turns it into a window operation instead of a regular call.
- `PARTITION BY region` means ranking restarts for each region.
- `ORDER BY amount DESC` means the highest amount gets rank 1.
- Every original row is preserved — nothing collapses.

> **RANK vs. DENSE_RANK vs. ROW_NUMBER:** With two tied amounts at rank 1, RANK skips to 3. DENSE_RANK would continue to 2. ROW_NUMBER would arbitrarily assign 1 and 2 — it never produces ties.

PRACTICE

Fill in the Blanks

Each exercise presents a scenario and a window function query with missing pieces. Type the correct SQL in each blank, then check your work. Use **Hint** if stuck, or **Reveal** to see the answer.

1 ROW_NUMBER — Assign Sequential Ranks

Easy

Assign a unique row number to each employee within their department, ordered by salary highest-first.

```
employees (id, name, department, salary, hire_date)
```

```
SELECT
    name, department, salary,
    ROW_NUMBER()  OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_rank
FROM employees;
```

function

keyword

partition clause

sort direction

Check Answer | Hint | Reveal | Reset

✓ All correct — nice work!

`ROW_NUMBER()` assigns a unique sequential integer starting at 1 within each partition. `OVER` activates the window, `PARTITION BY` resets numbering per department, and `DESC` ensures the highest salary gets 1.

2 Running Total with SUM() OVER

Easy

Calculate a cumulative running total of daily revenue, ordered by date. Each row should show its own amount plus all previous days' amounts.

```
daily_revenue (id, revenue_date, amount)
```

```
SELECT
    revenue_date,
    amount,
    SUM(amount) OVER (
        ORDER BY revenue_date ASC
    ) AS running_total
FROM daily_revenue;
```

aggregate fn

ordering clause

Check Answer | Hint | Reveal | Reset

✓ All correct — nice work!

When you put `SUM()` inside `OVER()` with an `ORDER BY`, it becomes a running (cumulative) sum. Without `PARTITION BY`, the entire table is one partition. Each row sees the sum of all amounts from the first row up to and including itself.

3 LAG — Compare to the Previous Row

Easy

For each region, show the current month's revenue alongside the *previous* month's revenue, then compute the month-over-month change.

```
monthly_sales (id, sale_month, region, revenue)
```

```
SELECT
    sale_month, region, revenue,
    LAG(revenue, 1) OVER (
        PARTITION BY region
        ORDER BY sale_month
    ) AS prev_month_rev,
    revenue - LAG(revenue, 1) OVER (
        PARTITION BY region
        ORDER BY sale_month
    ) AS mom_change
FROM monthly_sales;
```

function: LAG
offset: 1
partition col: REGION
order col: SALE_MONTH

Check Answer | Hint | Reveal | Reset

✓ All correct — nice work!

`LAG(revenue, 1)` reaches back one row (in the defined order) and returns that row's revenue. Subtracting it from the current revenue gives month-over-month change. The first row per partition returns NULL since there's no prior row.

4 Top-N per Group with DENSE_RANK

Medium

Find the **top 3 best-selling products** in each category. Use `DENSE_RANK()` so ties don't consume extra slots. Since you can't filter window results in WHERE, wrap it in a CTE.

```
products (id, name, category, units_sold)
```

```
WITH ranked_products AS (
    SELECT
        name, category, units_sold,
        DENSE_RANK() OVER (
            PARTITION BY category
            ORDER BY UNITS_SOLD DESC
        ) AS sales_rank
    FROM products
)
SELECT name, category, units_sold, sales_rank
FROM RANKED_PRODUCTS
WHERE sales_rank <= 3;
```

function

order col

CTE name

filter operator

Check Answer | Hint | Reveal | Reset

✓ All correct — nice work!

`DENSE_RANK()` ensures that if two products tie at rank 2, the next product is still rank 3 (not 4). The CTE wrapping is necessary because window functions can't be used in `WHERE` — they execute after filtering. The outer query filters `sales_rank <= 3` .

5 Moving Average with a Frame Clause

Medium

Calculate a **7-day moving average** of closing price for each stock ticker. The frame should include the current row and the 6 preceding rows.

```
stock_prices (id, ticker, trade_date, close_price)
```

```
SELECT
    ticker, trade_date, close_price,
    AVG                                    (close_price) OVER (
                                           PARTITION BY ticker
                                           ORDER BY trade_date
                                           ROWS                  6 PRECEDING                  AND CURRENT ROW
    ) AS moving_avg_7d
FROM stock_prices;
```

function — frame type — frame start

Check Answer | Hint | Reveal | Reset

Answer revealed. Study the pattern, then try the next one!

The frame clause `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` tells the database to compute the AVG using exactly 7 rows: the current row plus the 6 before it. `ROWS` counts physical rows; `RANGE` would compare values instead. This is the standard pattern for N-period moving averages.

6 NTILE — Percentile Buckets

Medium

Divide all customers into **4 equal quartile buckets** based on their total spending (highest spenders in bucket 1). Then show only the top quartile.

```
customers (id, name, total_spend, signup_date)
```

```
WITH bucketed AS (
    SELECT
        name, total_spend,
        NTILE(4) OVER (
            ORDER BY total_spend DESC
        ) AS spend_quartile
    FROM customers
)
SELECT name, total_spend, spend_quartile
FROM bucketed
WHERE spend_quartile = 1;
```

NTILE = function · 4 = bucket count · DESC = sort direction · 1 = filter value

Check Answer | Hint | Reveal | Reset

✓ All correct — nice work!

`NTILE(4)` splits the ordered result set into 4 roughly equal groups numbered 1–4. With `DESC` ordering, the highest spenders land in bucket 1. This is commonly used for percentile analysis, customer segmentation, and cohort identification.

7 Multi-Function Window — Prev, Next & Running Total

Hard

For each patient, on each visit show: the *previous* visit's charge, the *next* visit's charge, and the running total of all charges to date. Three different window functions, all partitioned by patient and ordered by visit date.

```
patient_visits (id, patient_id, visit_date, charge_amount)
```

```
SELECT
    patient_id, visit_date, charge_amount,

    LAG(charge_amount, 1) OVER w AS prev_charge,
    LEAD(charge_amount, 1) OVER w AS next_charge,
    SUM(charge_amount)      OVER w AS running_total

FROM patient_visits
WINDOW w AS (
    PARTITION BY patient_id
    ORDER BY visit_date
);
```

LAG — look-back fn
LEAD — look-ahead fn
WINDOW — define keyword
patient_id — partition col
visit_date — order col

Check Answer | Hint | Reveal | Reset

Answer revealed. Study the pattern, then try the next one!

The `WINDOW` clause (SQL standard, supported in PostgreSQL, MySQL 8+, SQLite 3.28+) lets you define a named window spec once and reference it with `OVER w` from multiple functions. This avoids repeating the same `PARTITION BY ... ORDER BY ...` three times. `LAG` looks back, `LEAD` looks forward, and `SUM` with an ordered window produces a running total.

SQL Window Functions Tutorial · Every row gets a window seat.